

AO 对接配合清单(给付卓新)

AO 对接配合清单(给测试侧 · 付卓新)

回应《AO Neo4j 接入对齐讨论与信息请求》。下面每一条都是 2026-06-23 在 AO 机器上实测(真发了 curl / 读了代码)后给的结论,不是文档推断。一句话定调:不用再搭新网络;方案直接走"路线 B(HTTP :3500)";只有 1 个必须改的点(IP)+ 2 个会绊到你的坑(测试 id 失效、关系不是图边)。

0. 先回答你最关心的两件事

网络:不需要共同连新网络、不需要 VPN/堡垒机。你已经能打通 :3002 执行 API,而 :3500 本体 API 和它是同一台机器、同一网段、只是换端口—— :3500 绑在 *:3500 (所有网卡,实测),所以你能到 :3002 就一定能到 :3500。

唯一必须改:旧接入卡写的 192.168.1.104 已失效(ping 100% 丢包,根本不是本机网卡)。AO 这台机器现在是 192.168.1.111。把你 env 里 ontology / execution 两个 base URL 的 IP 都改成 .111。

AO 侧会把这台机器固定成静态 IP,免得 DHCP 再漂(这就是 .104 变 .111 的根因)。

方案:路线 A(bolt 直连)整条放弃。原因:AO 自己没有独立 Neo4j,AO 真 Agent 读本体本来就只走 :3500 这个 HTTP API。所以你走 :3500(路线 B)读到的,就是真 Agent 跑用例时看到的同一份图——这正是你要的"单一数据源 + 真实可比对",且零新增基建。bolt 那层的 Neo4j 属于陈洋的 Allmeta Studio,AO 没法单方面开,而且走它反而和真 Agent 的取数路径不一致。

1. 连接参数(实测可用)

服务	Base URL	鉴权 header
本体 API(:3500,Allmeta	http://192.168.1.111:3500	Authorization: Bearer abc123

服务	Base URL	鉴权 header
Studio,背靠 Neo4j)		
执行 API(:3002)	http://192.168.1.111:3002/api/agent-execution	Authorization: Bearer aek_live_5f3c9b1e7a2d4f86b0c1e9d27a4f8c30

- 两把 token 不通用(实测): abc123 发到 :3002 → 401; aek_live_... 发到 :3500 → 401。各用各的。
- domain 含中文,必须 percent-encode:curl 用 -G --data-urlencode "domain=招聘-v1" 。 RAAS-v1 / raas 是历史别名,统一以 招聘-v1 为准。
- 你 env 直接这么填:

```
AO_ONTOLOGY_BASE_URL = http://192.168.1.111:3500
ALLMETA_API_KEY      = abc123
AGENT_EXECUTION_BASE = http://192.168.1.111:3002/api/agent-execution
AGENT_EXECUTION_API_KEY = aek_live_5f3c9b1e7a2d4f86b0c1e9d27a4f8c30
AO_ONTOLOGY_VERSION  = rules_v0_1_003 # 见 §5 的重要说明
```

⚠️ abc123 是个很弱的开发 token(归陈洋的 Studio .env 管)。AO 侧已在排期轮换;轮换后会连同静态 IP 一起给你一版干净接入卡。

2. 七个本体端点 —— 全部实测 200(附可直接粘的 curl)

域全部用 --data-urlencode "domain=招聘-v1",token -H "Authorization: Bearer abc123",下面省略。

#	端点	实 测	返回形状 / 要点
1	GET /api/v1/ontology/rules?limit=1000	200	{items:[Rule], nextCursor}。全集条, limit=1000 一页拉完(nextCursor

#	端点	实测	返回形状 / 要点
			就是你的覆盖率分母来源
2	GET /api/v1/ontology/rules/{id} (如 10-5)	200	单个 Rule(无 items 包裹)。 standardizedLogicRule = 你做 ruleTextSnapshot.sha256 的那段原
3	GET /api/v1/ontology/actions/ruleCheckForMatchResume/rules	200	比名字更全:整个 Action + action_steps rules[] + ruleCount + userPrompt Action→Step→Rule 链
4	GET /api/v1/ontology/instances/{Label}/{id}	200	单实例。注意 id 格式见 §4(旧卡的 C-2 001 已失效)
5	GET /api/v1/ontology/instances/{Label}?<filter>=<nextCursor>	200	{items, nextCursor}。任意属性都能 &candidate_id=C_5each369 实测精确
6	GET /api/v1/ontology/links?from=<id>&type=<REL>	200	{items, nextCursor}。当前 domain items:[] —— 见 §6,关系不是图边
7	GET /api/v1/ontology/schema/rules	200	{label, resource, sampledNodes:100 [{name, occurrence}]}, 15 个字段

翻页:列表端点(rules/instances/links)的 nextCursor 是 {"skip":N} 的 base64(eyJza2lwIjozfQ = {"skip":3})。非 null 时回传 &cursor=<nextCursor> 取下一页,null 即末页。单对象端点(单规则/单实例/action 链/schema)没有 cursor。

错误形状(都是干净 JSON,字段 error + message): 401 unauthorized (token 缺/错) · 400 missing-domain · 404 instance-not-found / node-not-found · 502 neo4j-unavailable。

数据小坑:Neo4j 时间字段(updatedAt 等)序列化成嵌套 {year:{low,high},month:{low,high},...},读 .low,不是 ISO 串(同一节点里 snapshot_at 这种又是普通 ISO 串,混着来)。

```
# 1 全集(覆盖率分母):实测 313 条一页拉完
curl -s -m10 -G "http://192.168.1.111:3500/api/v1/ontology/rules" \
  --data-urlencode "domain=招聘-v1" --data-urlencode "limit=1000" \
  -H "Authorization: Bearer abc123"

# 3 Action→Step→Rule 链(对账 retrievedRuleIds 用)
curl -s -m10 -G "http://192.168.1.111:3500/api/v1/ontology/actions/ruleCheckForMatchResume/rules" \
  --data-urlencode "domain=招聘-v1" -H "Authorization: Bearer abc123"
```

3. 逐条回应你的 5 个请求(①—⑤)

① 完整端点手册 —— 给你

随这份一起发你: docs/action_object_prompt/ONTOLOGY-API-USER-GUIDE-BASED-ON-NEO4J.md (1427 行,含全部端点 + 请求参数 + 响应 schema + 翻页约定)。你之前只收到接入卡,没收到这份。

② 两个端口的 token —— 不是同一把

- :3002 执行 API = `æk_live_5f3c9b1e7a2d4f86b0c1e9d27a4f8c30` (你已生效)
- :3500 本体 API = `abc123` (新给你的,见 §1)
- 实测互相串用都会 401。两套独立鉴权域。

③ 用例 id 存在性核对 —— 两个都已失效,给你替身

实测(本体 Neo4j):

- `Candidate / C-20260424-001` → 404 instance-not-found
- `Job_Requisition / R2026031629581` → 404 instance-not-found

已验证可用的替身 id: | Label | 替身 id | 说明 | |---|---|---| | Candidate | `C_5eachb369` | 周七 / Zhou Qi(单查 200) | | Job_Requisition | `mock_jr_v0_1_010_1778766302725` | 商务 BD 北京(单查 200) |

- candidate id 真实格式是 `C_<hex>` / `mock_cand_*` / `test_ping`,不是 `C-YYYYMMDD-NNN`;节点上 id 字段名是 `candidate_id` (不是 `id`),岗位是 `job_requisition_id`。
- 当前 `招聘-v1` 的实例都是 `mock_*` / `smoke_*` / `*_TEST*` / `e2e_*` 这类会被重置的数据,没有长期稳定的固定 fixture。AO 侧 action item:种一组永不删的标准测试三元组(Candidate+Job_Requisition+Application),给你一个能写死的 id。在那之前请用上面的替身,并做好 id 可能再变的准备。
- 提醒:这两个 id 在执行 API(:3002)那边是按 store ref 读的(可能走 partner-pg,不一定是本体 Neo4j)。我这次只确认了它们在本体 Neo4j 里没了;执行侧的验收用例(指南 §10 tc-de5ec703)我没实跑,你那边要用请单独确认 store 里还在不在。

④ 域名规范 —— 统一用 `招聘-v1`

`RAAS-v1` / `raas` 都解析到它,但请你代码里统一到 `招聘-v1` 为权威名。

⑤ 版本 pin —— 有个重要的"指纹≠执行"区别,务必看

- 可 pin 的版本只有两个(实测 `/capabilities`): `rules_v0_1_002` (248 条,meta 0.2)、`rules_v0_1_003` (261 条,meta 0.3)。

- 执行请求里 `ontology.version` 是必填,AO 没有默认值;填空/填未知值会硬报 `ONTOLOGY_VERSION_UNAVAILABLE` (不可重试)。请显式填 `rules_v0_1_003` (最新)。
- 关键区别(这条直接影响你的覆盖率分母):目前的 version pin 只用于校验 + 指纹,真正的 matchResume 判定是跑在 LIVE Neo4j 上的——当前 LIVE 有 313 条规则,比 261 的快照多 52 条。也就是说 pin `rules_v0_1_003` 并不代表引擎只评 261 条;它评的是 313 条的实时图。AO 在响应里用 `meta.ontologyLoaded.liveConsistency` 暴露这个差异。
- 所以:
 - 你要对齐"AO 实际评估面"的覆盖率分母 → 用 :3500 实时全集(313 条,再按 `executor='Agent'` 且 `enforcementLevel='mandatory'` 自筛),不要用 261 的快照数。
 - 261/248 的快照只是用于指纹复现。
 - :3500 的 `/rules` 不支持 `&version=` 过滤(实测带上返回 0 条);要拿冻结的 248/261 条原文得用 AO 的 snapshot 文件,不能从 live API 取。
 - AO 侧 action item:把 live(313)和最新快照(261)对齐(重生成 `rules_v0_1_004` 或修剪 live),消除这个 52 条的缺口。

4. 你那 5 类图遍历 Cypher 的 HTTP 覆盖结论

读完整手册 + 实测后,本体 API 是"Neo4j 之上的 CRUD",不是图遍历层(手册"What's not in v1"明说:无 Workflow CRUD、无裸 Cypher、无路径接口)。逐条:

#	你的 Cypher	结论	替代 / 说明
1	<code>shortestPath(...)</code>	无对应 (GAP)	没有 path/shortestPath/graph 端点(实测全 404)
2	<code>[[:CHECKS GOVERNS*0..]]</code> 规则→下游消费者	部分(只反向)	只有 action 视角: <code>GET /actions/{ref}/rules</code> 内部走 Action→Step→Rule。要"给定规则查谁用它",得枚举 <code>/actions</code> 再逐个调 <code>/rules</code> 反查;没有 <code>/rules/{id}/consumers</code>
3	<code>(:Workflow)-[:RUNS_ACTION]->(:Action)</code>	无对应 (GAP)	根本没有 workflow 端点(<code>/workflows</code> →404),Workflow 在本 API 里不是建模实体
4	<code>(:Client)-[*1..2]-(:Job_Requisition)</code>	无对应 (GAP)	没有变长遍历。只能客户端按 FK 字段自己 join(见 §6);且当前 招聘-v1 的 Client 实例为 0 行,连锚点都没有
5	<code>type(rel)</code> 关系类型发现	无对应 (GAP)	<code>/schema</code> 只做节点属性自省; <code>/links?type=</code> 一次只能验一个类型

你漏掉的 **bonus** 端点(建议补进客户端): `GET /objects /actions /events` (四类节点通用 CRUD,枚举 Action 给 #2 用)· `GET /schema` (一次拿四类 label,不只 `/schema/rules`)· `GET /actions/{ref}/steps` (只要步骤不要规则)· 完整 `/links` 的 POST/PUT/PATCH/DELETE · `POST /actions/matchResume/results` (单 action 写结果)。这些都不增加路径/变长遍历能力。

5. 关系不是图边 —— 这条会改变你的实现(重要)

实测: `/links` 对当前 domain 里所有节点(失效候选人、两个活候选人、活岗位)都返回 `items:[]`;不带 `from` 也是空。也就是说 **招聘-v1** 里的业务关系不是存成 Neo4j 边,而是存成节点上的外键字段——例如 **Application** 节点带 `candidate_id` + `job_requisition_id`。

对你的影响:

- 候选人↔岗位的遍历(你 L3 grounding 那块),不要等图边,改成:读 **Application** / 节点实例的 prop bag,在你自己代码里按 `candidate_id` / `job_requisition_id` 等 FK 属性 join。
- **EMPLOYED_BY** 这类 `/links` 调用现在拿不到数据,别依赖。
- AO 侧 action item:确认 **招聘-v1** 到底是"边没灌"还是"模型本就用 FK 不用边",并在接入卡里写清,免得你按图遍历预期去对接。

6. 执行 API(:3002)对接契约(你已接通,这里补权威细节)

- 异步 shadow 模式: `POST /executions` → 202 + `executionId` → 轮询 `GET /executions/{id}` 到 `status=succeeded/failed` (2-5s 一次,单用例 ≤3 分钟)→ 需要全文再 `GET /executions/{id}/llm-calls/{callId}`。 **shadow** 是唯一模式(判定真跑,副作用全抑制并在 `result.emittedEvents` / `trace.steps[].suppressedSideEffects` 申报)。
- liveness 探针 = `GET /capabilities` (返回 JSON)。别拿裸 `GET /api/agent-execution` 当健康检查(返回的是 Next 应用 HTML,不是 JSON);没有 `/health`、`/ping`。
- join key —— 你最关心的对账: `trace.retrievedRuleIds[]` 和 `ruleEvaluations[].ruleId` 就是本体原始 id(如 **10-5**),端到端零改写,和 :3500 `/rules` 返回的 id 完全一致 → 直接字符串等值 join,安全。
- 并发: `maxConcurrent=4` (实测)。超了 → `429` + **Retry-After**,body `{error.code:RATE_LIMITED, retryable:true}`,按 **Retry-After** 退避。
- 幂等: `clientRequestId` 是必填幂等键,同 id 永远返回同 `executionId`、不重跑(≥7 天);结果保留 7 天。
- 覆盖率口径对齐:AO 的评估面只含 `executor='Agent'` 且 `enforcementLevel='mandatory'` 的规则——你算覆盖率时用同样的筛选口径。

- 两个状态语义差异(和你 5 态模型不同,别合并): `ruleEvaluations[].status` 有第 6 态 `evaluated_inconclusive` (它才是 `decision=needs_review` 的来源); `retrieved_not_evaluated` 在 AO 指"被确定性预筛排除、从未进 LLM 上下文"(单 pass),和你"进了上下文但没走到"的定义不一样。

⚠ 稳定性:实测第一发 curl 撞到一次 connection-refused(Next dev server 正在热重载),随后连续 200。你大批量发用例(30—60 条)前,请 AO 确认 :3002 是用 `npm run start` (常驻)而不是热重载的 dev server 起的,否则中途重载会变成零星 connection-refused(不是能干净重试的 4xx/5xx)。

7. AO 侧待办(给 AO 团队,不用你跟进)

1. 把这台机器固定静态 IP(现 `.111` 是 DHCP)——根治 base URL 漂移。
2. 在 `招聘-v1` 种一组永不删的标准测试三元组(Candidate+Job_Requisition+Application),给测试侧一个能写死的 id;同步更新接入卡 / 指南 §10 里失效的 `C-20260424-001` / `R2026031629581`。(接入卡里的 IP+示例 id 已于 2026-06-23 先行更正。)
3. 对齐 live(313)与最新快照(261):重生成 `rules_v0_1_004` 或修剪 live;并在执行响应里把 `liveConsistency` (如 live 313 vs pinned 261)落成具体数字。
4. 决定 version pin 的真实语义:要么实现"按版本加载规则",要么明确标注"版本=指纹/校验,判定跑 live"。
5. 跟陈洋一起轮换 :3500 的弱 token `abc123`;轮换后连静态 IP 出一版干净接入卡。6.(可选)评估 5 个图遍历 GAP 是否要补端
点: `/path` (shortestPath)、`/workflows` + `/workflows/{ref}/actions`、`/instances/{label}/{value}/neighbors?hops=1..2`、`/schema/relationships`、`/rules/{id}/consumers`。
6. 确认 :3500/:3002 长期是否要全网卡暴露,还是限到 LAN 子网/partner host。